ASSIGNMENT 1 - COMPARATIVE ANALYSIS OF OPTIMIZATION ALGORITHMS

23CCE215 - MACHINE LEARNING

TEAM MEMBERS:

HAMSHITHA P - CB.EN.U4CCE24115

JOTHSNA KISHORE - CB.EN.U4CCE24119

QUESTION:

Select a function of two variables, say x and y, given by f(x,y). The function should have at least one local minimum and one global minimum.

Find the minimum of the given function using the following algorithms:

a. Gradient Descent

b. Gradient Descent with Momentum

c. Adagrad

Select the parameters as per your understanding and state them clearly at the beginning of the algorithm. Show the first three iterations only.

Now implement each of these algorithms using MATLAB Live-script. Use separate live scripts for each algorithm. DO NOT use any built-in functions. State clearly the termination condition used for each code example. Add detailed text explanations of the code sections to improve readability.

Use appropriate visualization to illustrate how the surface of the function f(x,y) looks like, the initial starting point and the movement of the solution in the direction of the global optimum.

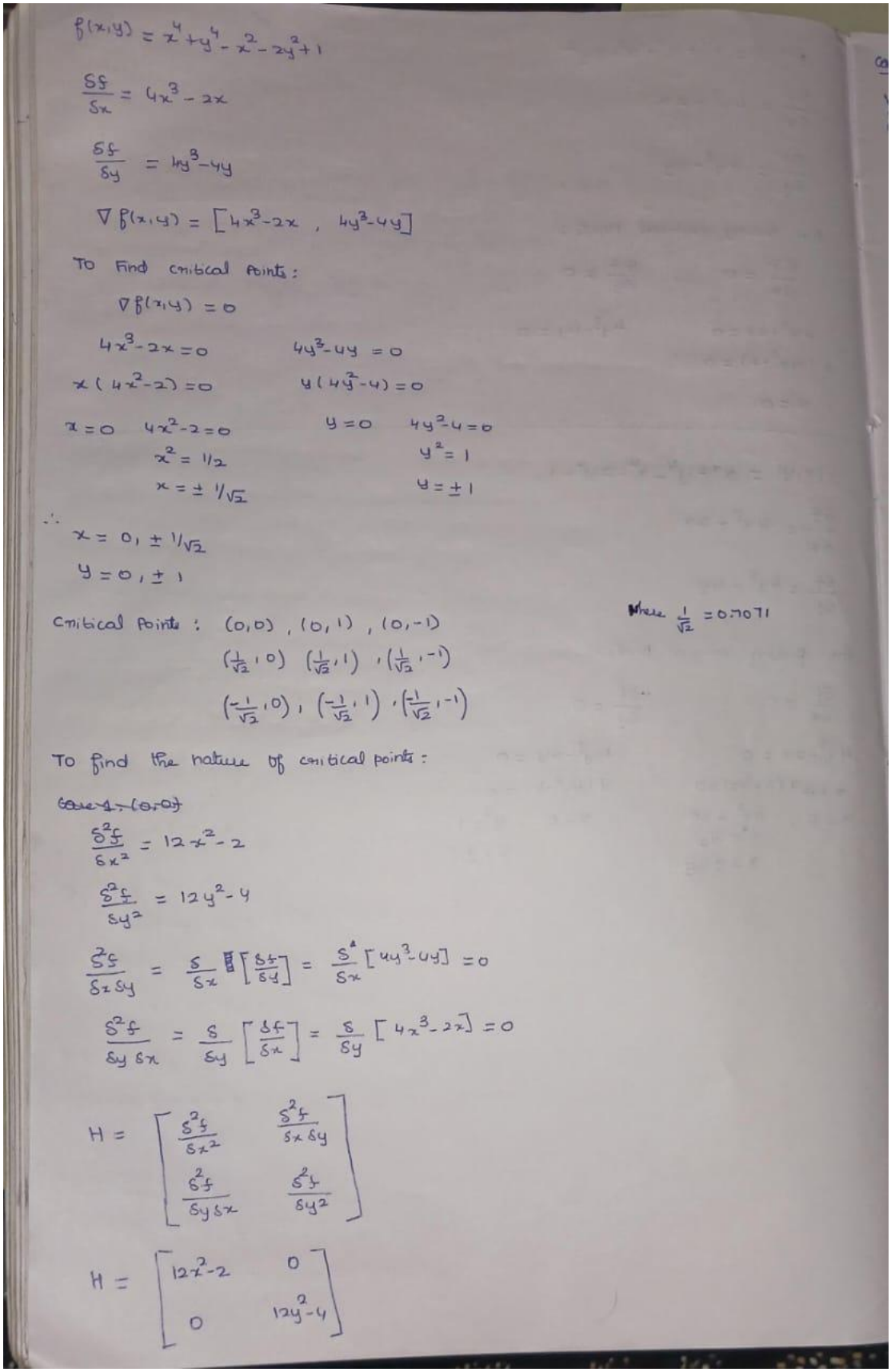
FUNCTION USED:

$f(x,y)=x^4+y^4-x^2-2*y^2+1$

The function has four local minima located at $(\pm 0.7071, \pm 1)$.
Since the function attains the same minimum value $f_{min} = -0.25$ at each of these points and no lower value exists elsewhere, all four local minima are also global minima.

MANUAL CALCULATION:

1. Critical points



$f(x,y) = x^4 + y^4 - x^2 - 2y^2 + 1$

$\frac{\delta f}{\delta x} = 4x^3 - 2x$

$\frac{\delta f}{\delta y} = 4y^3 - 4y$

$\nabla f(x,y) = [4x^3 - 2x \;,\; 4y^3 - 4y]$

To Find critical points:

$\nabla f(x,y) = 0$

$4x^3 - 2x = 0$ $\qquad$ $4y^3 - 4y = 0$

$x(4x^2 - 2) = 0$ $\qquad$ $y(4y^2 - 4) = 0$

$x = 0$ $\quad 4x^2 - 2 = 0$ $\qquad$ $y = 0$ $\quad 4y^2 - 4 = 0$

$x^2 = 1/2$ $\qquad$ $y^2 = 1$

$x = \pm 1/\sqrt{2}$ $\qquad$ $y = \pm 1$

$\therefore$

$x = 0, \pm 1/\sqrt{2}$

$y = 0, \pm 1$

Where $\frac{1}{\sqrt{2}} = 0.7071$

Critical Points : $(0,0)$, $(0,1)$, $(0,-1)$

$\left(\frac{1}{\sqrt{2}}, 0\right)$ $\left(\frac{1}{\sqrt{2}}, 1\right)$, $\left(\frac{1}{\sqrt{2}}, -1\right)$

$\left(\frac{-1}{\sqrt{2}}, 0\right)$, $\left(\frac{-1}{\sqrt{2}}, 1\right)$, $\left(\frac{-1}{\sqrt{2}}, -1\right)$

To find the nature of critical points :

~~Case 1 : (0,0)~~

$\frac{\delta^2 f}{\delta x^2} = 12x^2 - 2$

$\frac{\delta^2 f}{\delta y^2} = 12y^2 - 4$

$\frac{\delta^2 f}{\delta x \delta y} = \frac{\delta}{\delta x}\left[\frac{\delta f}{\delta y}\right] = \frac{\delta}{\delta x}[4y^3 - 4y] = 0$

$\frac{\delta^2 f}{\delta y \delta x} = \frac{\delta}{\delta y}\left[\frac{\delta f}{\delta x}\right] = \frac{\delta}{\delta y}[4x^3 - 2x] = 0$

$$H = \begin{bmatrix} \frac{\delta^2 f}{\delta x^2} & \frac{\delta^2 f}{\delta x \delta y} \\ \frac{\delta^2 f}{\delta y \delta x} & \frac{\delta^2 f}{\delta y^2} \end{bmatrix}$$

$$H = \begin{bmatrix} 12x^2 - 2 & 0 \\ 0 & 12y^2 - 4 \end{bmatrix}$$

Case 1: CP (0,0)

$H @ (0,0) = \begin{bmatrix} -2 & 0 \\ 0 & -4 \end{bmatrix}$

$|H - \lambda I| = \begin{vmatrix} -2-\lambda & 0 \\ 0 & -4-\lambda \end{vmatrix} = 0$

$(-2-\lambda)(-4-\lambda) - 0 = 0$

$(\lambda+2)(\lambda+4) = 0$

$\lambda = -2, -4$

Eigen values are -ve.

∴ Critical point (0,0) → local maxima

Case 2: CP: (0,1)

$H @ (0,1) = \begin{bmatrix} -2 & 0 \\ 0 & 8 \end{bmatrix}$

$|H - \lambda I| = \begin{vmatrix} -2-\lambda & 0 \\ 0 & 8-\lambda \end{vmatrix} = 0$

$(-2-\lambda)(8-\lambda) = 0$

$(2+\lambda)(8-\lambda) = 0$

$16 - 2\lambda + 8\lambda - \lambda^2 = 0$

$-\lambda^2 + 6\lambda + 16 = 0$ $\quad \lambda = 8, -2$

∴ Eigen values have both +ve & -ve

∴ Critical Point (0,1) is saddle point

Case 3: CP: (0,-1)

$H @ (0,-1) = \begin{bmatrix} -2 & 0 \\ 0 & 8 \end{bmatrix}$

∴ $\lambda = 8, -2$

Critical point (0,-1) → saddle point.

Case 4: CP: $\left(\frac{+1}{\sqrt{2}}, 0\right)$

$H @ \left(\frac{1}{\sqrt{2}}, 0\right) = \begin{bmatrix} 4 & 0 \\ 0 & -4 \end{bmatrix}$

$|H - \lambda I| = \begin{vmatrix} 4-\lambda & 0 \\ 0 & -4-\lambda \end{vmatrix} = 0$

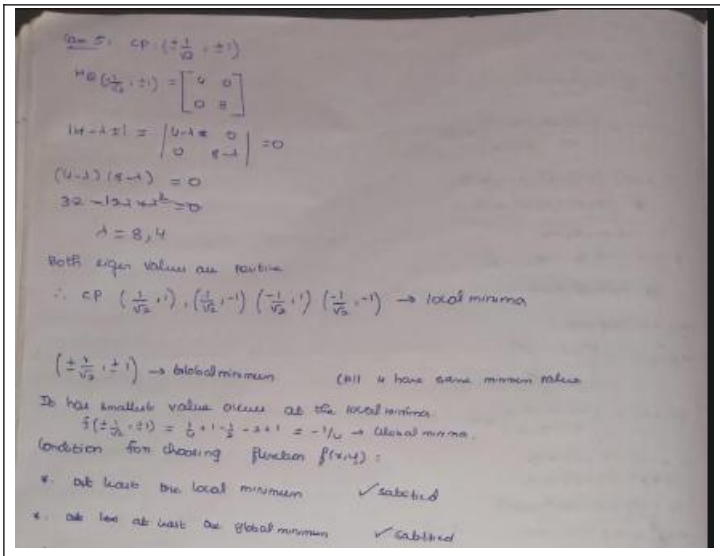
$(4-\lambda)(-4-\lambda) = 0$

$(4-\lambda)(4+\lambda) = 0$

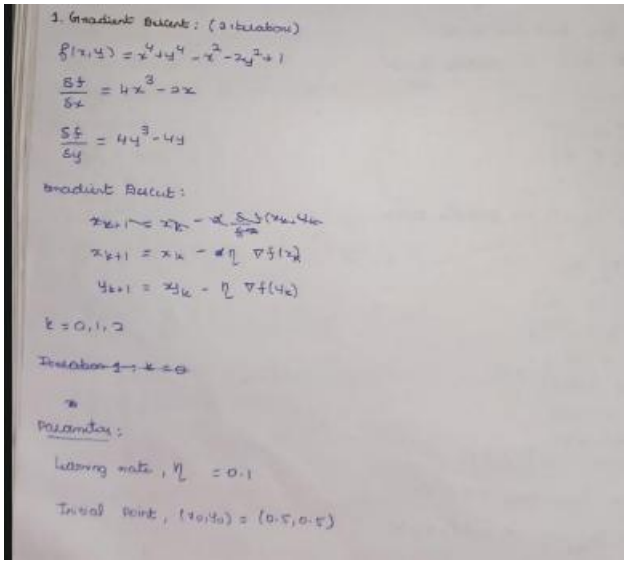
$16 - \lambda^2 = 0 \Rightarrow \boxed{\lambda = \pm 4}$

∴ Eigen values have both +ve & -ve

Critical point $\left(\pm\frac{1}{\sqrt{2}}, 0\right)$ → saddle point



2. Gradient descent



Iteration 1 : k=0

$(x_0, y_0) = (0.5, 0.5)$

$\nabla f(x,y) = [4x0.5^3 - 2x0.5 \;,\; 4x0.5^3 - 4x0.5]$

$\nabla f(x,y)_{@(0.5,0.5)} = [-0.5, -1.5]$

$\nabla f(x,y)_{@(x_0,y_0)} = [-0.5, -1.5]$

$x_1 = x_0 - \eta \nabla f(x_0)_{x_0=0.5}$

$y_1 = y_0 - \eta \nabla f(y_0)_{y_0=0.5}$

$\therefore x_1 = 0.5 - 0.1 \times -0.5 = 0.55$

$y_1 = 0.5 - 0.1 \times -1.5 = 0.65$

$(x_1, y_1) = (0.55, 0.65)$

Iteration 2 : k=1

$(x_1, y_1) = (0.55, 0.65)$

$\nabla f(x,y)_{@(x_1,y_1)=(0.55,0.65)} = [-0.4345 \;\; 4x0.55^3 - 2x0.55,\; 4x0.65^3 - 4x0.65]$

$= [-0.4345, -1.5015]$

$x_2 = x_1 - \eta \nabla f(x_1)$

$y_2 = y_1 - \eta \nabla f(y_1)$

$x_2 = 0.55 - 0.1 \times -0.4345 = 0.59345 \simeq 0.5935$

$y_2 = 0.65 - 0.1 \times -1.5015 = 0.80015 \simeq 0.8002$

$(x_2, y_2) = (0.5935, 0.8002)$

Iteration 3 : k=2

$(x_2, y_2) = (0.5935, 0.8002)$

$\nabla f(x,y)_{@(x_2,y_2)=(0.5935,0.8002)} = [4x(0.5935)^3 - 2x0.5935,\; 4x(0.8002)^3 - 4x0.8002]$

$= [-0.35077, -1.15126]$

$\simeq [-0.3508, -1.1513]$

$x_3 = x_2 - \eta \nabla f(x_2)$

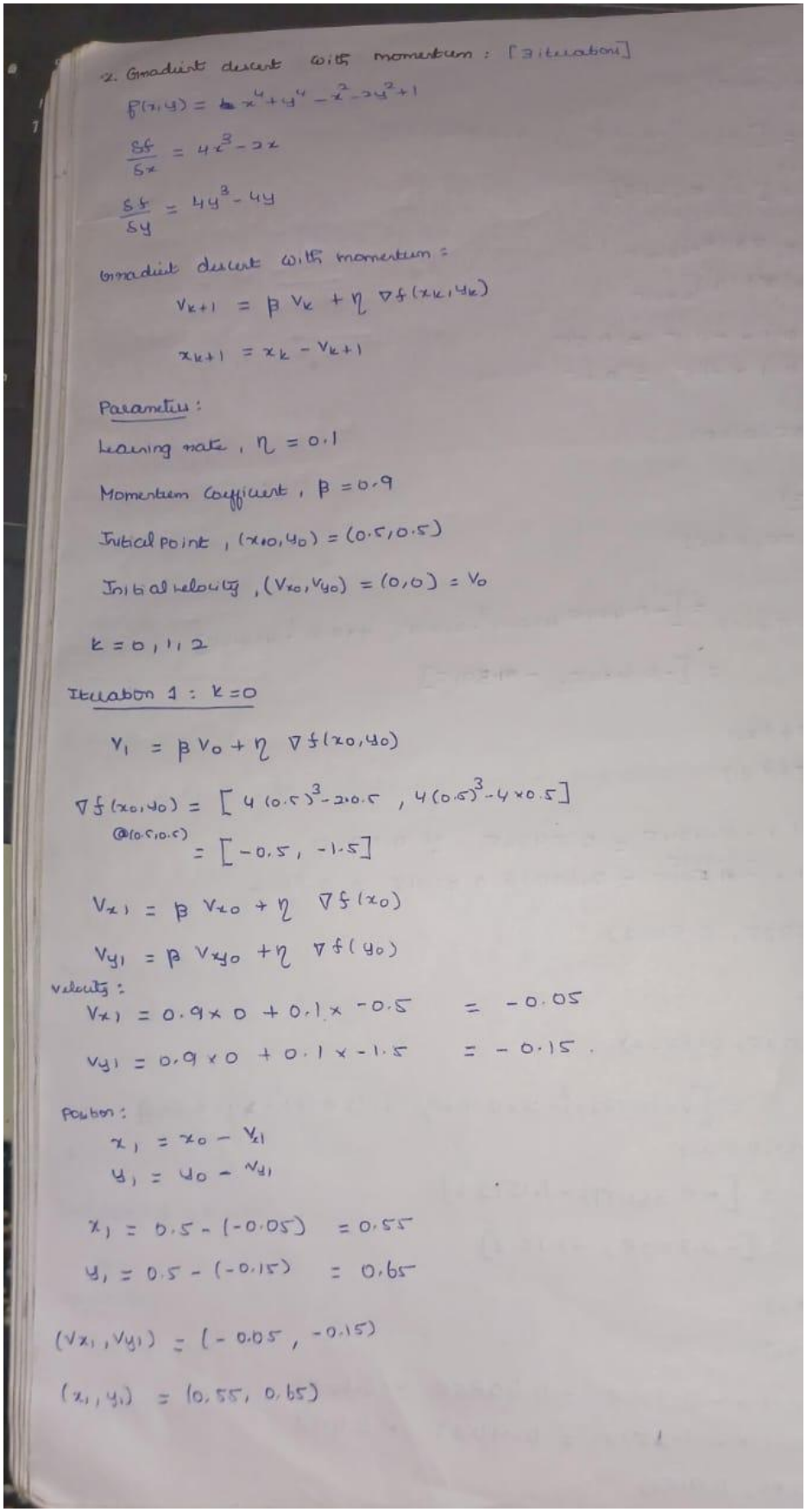
$y_3 = y_2 - \eta \nabla f(y_2)$

$x_3 = 0.5935 - 0.1 \times -0.3508 = 0.62858 \simeq 0.6286$

$y_3 = 0.8002 - 0.1 \times -1.1513 = 0.91533 \simeq 0.9153$

$(x_3, y_3) = (0.6286, 0.9153)$

3. Gradient descent with momentum



2. Gradient descent with momentum : [3 iterations]

$f(x,y) = x^4 + y^4 - x^2 - 2y^2 + 1$

$\frac{\partial f}{\partial x} = 4x^3 - 2x$

$\frac{\partial f}{\partial y} = 4y^3 - 4y$

Gradient descent with momentum :

$v_{k+1} = \beta v_k + \eta \nabla f(x_k, y_k)$

$x_{k+1} = x_k - v_{k+1}$

Parameters :

Learning rate, $\eta = 0.1$

Momentum coefficient, $\beta = 0.9$

Initial point, $(x_0, y_0) = (0.5, 0.5)$

Initial velocity, $(v_{x0}, v_{y0}) = (0,0) = v_0$

$k = 0, 1, 2$

Iteration 1 : $k = 0$

$v_1 = \beta v_0 + \eta \nabla f(x_0, y_0)$

$\nabla f(x_0, y_0) = [4(0.5)^3 - 2 \cdot 0.5 \;,\; 4(0.5)^3 - 4 \times 0.5]$

@(0.5, 0.5) $= [-0.5, -1.5]$

$v_{x1} = \beta v_{x0} + \eta \nabla f(x_0)$

$v_{y1} = \beta v_{y0} + \eta \nabla f(y_0)$

velocity :

$v_{x1} = 0.9 \times 0 + 0.1 \times -0.5 = -0.05$

$v_{y1} = 0.9 \times 0 + 0.1 \times -1.5 = -0.15$

Position :

$x_1 = x_0 - v_{x1}$

$y_1 = y_0 - v_{y1}$

$x_1 = 0.5 - (-0.05) = 0.55$

$y_1 = 0.5 - (-0.15) = 0.65$

$(v_{x1}, v_{y1}) = (-0.05, -0.15)$

$(x_1, y_1) = (0.55, 0.65)$

Iteration 2 : k=1

$(v_{x1}, v_{y1}) = (-0.05, -0.15)$

$(x_1, y_1) = (0.55, 0.65)$

$v_2 = \beta v_1 + \eta \nabla f(x_1, y_1)$

$\nabla f(x_1, y_1) = [4 \times (0.55)^3 - 2 \times 0.55 \;,\; 4 \times 0.65^3 - 4 \times 0.65]$

@(0.55, 0.65) $= [-0.4345, -1.5015]$

velocity :

$v_{x2} = \beta v_{x1} + \eta \nabla f(x_1)$

$v_{y2} = \beta v_{y1} + \eta \nabla f(y_1)$

velocity : $v_{x2} = 0.9 \times -0.05 + 0.1 \times -0.4345 = -0.08845 \simeq -0.0885$

$v_{y2} = 0.9 \times -0.15 + 0.1 \times -1.5015 = -0.28515 \simeq -0.2852$

Position :

$x_2 = x_1 - v_{x2}$

$y_2 = y_1 - v_{y2}$

$x_2 = 0.55 - (-0.0885) = 0.6385$

$y_2 = 0.65 - (-0.2852) = 0.9352$

$(x_2, y_2) = (0.6385, 0.9352)$

$(v_{x2}, v_{y2}) = (-0.0885, -0.2852)$

Iteration 3 : k=2

$(v_{x2}, v_{y2}) = (-0.0885, -0.2852)$

$(x_2, y_2) = (0.6385, 0.9352)$

$\nabla f(x_2, y_2) = [4 \times (0.6385)^3 - 2 \times (0.6385) \;,\; 4 \times (0.9352)^3 - 4 \times (0.9352)]$

@(0.6385, 0.9352) $= [-0.23577, -0.46909]$

$\simeq [-0.2358, -0.4691]$

Velocity :

$v_{x3} = 0.9 v_{x2} + 0.1 \nabla f(x_2)$

$v_{y3} = 0.9 v_{y2} + 0.1 \nabla f(y_2)$

$v_{x3} = 0.9 \times -0.0885 + 0.1 \times -0.2358 = -0.10323 \simeq -0.1032$

$v_{y3} = 0.9 \times -0.2852 + 0.1 \times -0.4691 = -0.30859 \simeq -0.3036$

Position :

$x_3 = x_2 - v_{x3} = 0.6385 - (-0.1032) = 0.7417$

$y_3 = y_2 - v_{y3} = 0.9352 - (-0.3036) = 1.2388$

$(v_{x3}, v_{y3}) = (-0.1302, -0.3036)$

$(x_3, y_3) = (0.7417, 1.2388)$

4. ADAGRAD

3. ADAGRAD: [3 iterations]

$f(x,y) = x^4 + y^4 - x^2 - 2y^2 + 1$

$\nabla f(x,y) = [4x^3 - 2x \;,\; 4y^3 - 4y]$

ADAGRAD:

$$V_{k+1} = V_k + [g(x_k, y_k)]^2$$

$$x_{k+1} = x_k - \frac{\eta}{\epsilon + \sqrt{V_{x,k+1}}} g(x_k)$$

$$y_{k+1} = y_k - \frac{\eta}{\epsilon + \sqrt{V_{y,k+1}}} g(y_k)$$

Parameters:

Learning rate, $\eta = 0.8$

Small constant, $\epsilon = 10^{-8}$

Initial point, $(x_0, y_0) = (0.5, 0.5)$

Initial accumulator, $(V_{x0}, V_{y0}) = (0, 0)$

$k = 0, 1, 2$

Iteration 1: $k = 0$

$(x_0, y_0) = (0.5, 0.5)$

$(V_{x0}, V_{y0}) = (0, 0)$

$g(x_0, y_0) = \nabla f(x_0, y_0) = [4 \times (0.5)^3 - 2 \times 0.5 \;,\; 4 \times (0.5)^3 - 4 \times 0.5]$

$= [-0.5, -1.5]$

$V_1 = V_0 + [g(x_0, y_0)]^2$

$V_{x1} = V_{x0} + [g(x_0)]^2 = 0 + (-0.5)^2 = 0.25$

$V_{y1} = V_{y0} + [g(y_0)]^2 = 0 + (-1.5)^2 = 2.25$

Position: $x_{t+1} = x_0 - \frac{\eta}{\epsilon + \sqrt{V_{x1}}} g(x_0)$

$$x_1 = 0.5 - \frac{0.8}{10^{-8} + \sqrt{0.25}} \times -0.5 = 1.29999 \simeq 1.3$$

$$y_1 = y_0 - \frac{\eta}{\epsilon + \sqrt{V_{y1}}} g(y_0)$$

$$= 0.5 - \frac{0.8}{10^{-8} + \sqrt{2.25}} \times -1.5 = 1.29999 \simeq 1.3$$

$\therefore (V_{x1}, V_{y1}) = (0.25, 2.25)$

$(x_1, y_1) = (1.3, 1.3)$

Iteration 2 : $k=1$

$(x_1, y_1) = (1.3, 1.3)$

$(v_{x1}, v_{y1}) = (0.25, 0.25)$

$g(x_1,y_1) = \nabla f(x_1,y_1) = [4\times(1.3)^3 - 2\times 1.3,\ 4(1.3)^3 - 4\times 1.3]$

$= [6.188,\ 3.588]$

$v_2 = v_1 + [g(x_1,y_1)]^2$

$v_{x2} = v_{x1} + [g(x_1)]^2 = 0.25 + (6.188)^2 = 38.5413$

$v_{y2} = v_{y1} + [g(y_1)]^2 = 0.25 + (3.588)^2 = 15.1237$

$\therefore x_2 = x_1 - \frac{\eta}{\epsilon + \sqrt{v_{x2}}} g(x_1) = 1.3 - \frac{0.8 \times 6.188}{10^{-8} + \sqrt{38.5413}} \simeq 0.5026$

$y_2 = y_1 - \frac{\eta}{\epsilon + \sqrt{v_{y2}}} g(y_1) = 1.3 - \frac{0.8 \times 3.588}{10^{-8} + \sqrt{15.1237}} \simeq 0.5619$

$(v_{x2}, v_{y2}) = (38.5413, 15.1237)$

$(x_2, y_2) = (0.5026, 0.5619)$

Iteration 3 : $k=2$

$(x_2, y_2) = (0.5026, 0.5619)$

$(v_{x2}, v_{y2}) = (38.5413, 15.1237)$

$g(x_2,y_2) = \nabla f(x_2,y_2) = [4\times(0.5026)^3 - 2\times(0.5026),\ 4(0.5619)^3 - 4(0.5619)]$

$= [-0.4974,\ -1.5380]$

$v_3 = v_2 + [g(x_2,y_2)]^2$

$v_{x3} = v_{x2} + [g(x_2)]^2 = 38.5413 + (-0.4974)^2 = 38.7887$

$v_{y3} = v_{y2} + [g(y_2)]^2 = 15.1237 + (-1.5380)^2 = 17.4891$

$x_3 = x_2 - \frac{\eta}{\epsilon + \sqrt{v_{x3}}} g(x_2) = 0.5026 - \frac{0.8 \times -0.4974}{10^{-8} + \sqrt{38.7887}} \simeq 0.5665$

$y_3 = y_2 - \frac{\eta}{\epsilon + \sqrt{v_{y3}}} g(y_2) = 0.5619 - \frac{0.8 \times -1.5380}{10^{-8} + \sqrt{17.4891}} \simeq 0.8561$

$\therefore (v_{x3}, v_{y3}) = (38.7887, 17.4891)$

$(x_3, y_3) = (0.5665, 0.8561)$

MATLAB LIVESCRIPT:

1.Critical points

```
%% ============================================================
% OBJECTIVE FUNCTION
%% ============================================================
f = @(x,y) x.^4 + y.^4 - x.^2 - 2*y.^2 + 1;

%% ============================================================
% FIRST DERIVATIVES (GRADIENT)
%% ============================================================
dfx = @(x) 4*x.^3 - 2*x;
dfy = @(y) 4*y.^3 - 4*y;

%% ============================================================
% SECOND DERIVATIVES (HESSIAN COMPONENTS)
%% ============================================================
d2fxx = @(x) 12*x.^2 - 2;
d2fyy = @(y) 12*y.^2 - 4;
d2fxy = 0;

%% ============================================================
% CRITICAL POINTS
%% ============================================================
x_vals = [0, 1/sqrt(2), -1/sqrt(2)];
y_vals = [0, 1, -1];

fprintf('All Critical Points:\n');
critical_points = [];

for i = 1:length(x_vals)
    for j = 1:length(y_vals)
        critical_points = [critical_points; x_vals(i) y_vals(j)];
        fprintf('    (%.4f , %.4f)\n', x_vals(i), y_vals(j));
    end
end
```

```
All Critical Points:

  (0.0000 , 0.0000)
  (0.0000 , 1.0000)
  (0.0000 , -1.0000)
  (0.7071 , 0.0000)
  (0.7071 , 1.0000)
  (0.7071 , -1.0000)
  (-0.7071 , 0.0000)
  (-0.7071 , 1.0000)
  (-0.7071 , -1.0000)
```

```
%% ================================================================
% CONTAINERS FOR LOCAL MINIMA
%% ================================================================
local_min_points = [];
local_min_values = [];

%% ================================================================
% ANALYSIS OF EACH CRITICAL POINT
%% ================================================================
for k = 1:size(critical_points,1)

    x = critical_points(k,1);
    y = critical_points(k,2);

    fprintf('Critical Point (%.4f , %.4f)\n\n', x, y);

    %% Hessian matrix
    H = [ d2fxx(x),    d2fxy;
          d2fxy,       d2fyy(y) ];

    fprintf('Hessian Matrix:\n');
    fprintf('[ %.4f     %.4f ]\n', H(1,1), H(1,2));
    fprintf('[ %.4f     %.4f ]\n\n', H(2,1), H(2,2));

    %% Eigenvalues
    eig_vals = eig(H);
    fprintf('Eigenvalues of Hessian:\n');
    fprintf('[ %.4f , %.4f ]\n\n', eig_vals(1), eig_vals(2));

    %% Function value
    f_val = f(x,y);
    fprintf('Function value: %.4f\n\n', f_val);

    %% Classification with reason
    if all(eig_vals > 0)
        nature = 'Local Minimum';
        reason = 'Both eigenvalues are positive';
        local_min_points = [local_min_points; x y];
        local_min_values = [local_min_values; f_val];

    elseif all(eig_vals < 0)
        nature = 'Local Maximum';
        reason = 'Both eigenvalues are negative';

    elseif any(eig_vals > 0) && any(eig_vals < 0)
        nature = 'Saddle Point';
        reason = 'One eigenvalue is positive and the other is negative';

    else
        nature = 'Inconclusive';
        reason = 'At least one eigenvalue is zero';
    end
```

```
        fprintf('Nature : %s\n', nature);
        fprintf('Reason : %s\n\n', reason);
end
```

```
Critical Point (0.0000 , 0.0000)
Hessian Matrix:
[ -2.0000 0.0000 ]
[ 0.0000 -4.0000 ]
Eigenvalues of Hessian:
[ -4.0000 , -2.0000 ]
Function value: 1.0000
Nature : Local Maximum
Reason : Both eigenvalues are negative


Critical Point (0.0000 , 1.0000)
Hessian Matrix:
[ -2.0000 0.0000 ]
[ 0.0000 8.0000 ]
Eigenvalues of Hessian:
[ -2.0000 , 8.0000 ]
Function value: 0.0000
Nature : Saddle Point
Reason : One eigenvalue is positive and the other is negative


Critical Point (0.0000 , -1.0000)
Hessian Matrix:
[ -2.0000 0.0000 ]
[ 0.0000 8.0000 ]
Eigenvalues of Hessian:
[ -2.0000 , 8.0000 ]
Function value: 0.0000
Nature : Saddle Point
Reason : One eigenvalue is positive and the other is negative


Critical Point (0.7071 , 0.0000)
Hessian Matrix:
[ 4.0000 0.0000 ]
[ 0.0000 -4.0000 ]
Eigenvalues of Hessian:
[ -4.0000 , 4.0000 ]
Function value: 0.7500
Nature : Saddle Point
Reason : One eigenvalue is positive and the other is negative


Critical Point (0.7071 , 1.0000)
Hessian Matrix:
[ 4.0000 0.0000 ]
[ 0.0000 8.0000 ]
Eigenvalues of Hessian:
[ 4.0000 , 8.0000 ]
```

Function value: -0.2500
Nature : Local Minimum
Reason : Both eigenvalues are positive

Critical Point (0.7071 , -1.0000)
Hessian Matrix:
[4.0000 0.0000]
[0.0000 8.0000]
Eigenvalues of Hessian:
[4.0000 , 8.0000]
Function value: -0.2500
Nature : Local Minimum
Reason : Both eigenvalues are positive

Critical Point (-0.7071 , 0.0000)
Hessian Matrix:
[4.0000 0.0000]
[0.0000 -4.0000]
Eigenvalues of Hessian:
[-4.0000 , 4.0000]
Function value: 0.7500
Nature : Saddle Point
Reason : One eigenvalue is positive and the other is negative

Critical Point (-0.7071 , 1.0000)
Hessian Matrix:
[4.0000 0.0000]
[0.0000 8.0000]
Eigenvalues of Hessian:
[4.0000 , 8.0000]
Function value: -0.2500
Nature : Local Minimum
Reason : Both eigenvalues are positive

Critical Point (-0.7071 , -1.0000)
Hessian Matrix:
[4.0000 0.0000]
[0.0000 8.0000]
Eigenvalues of Hessian:
[4.0000 , 8.0000]
Function value: -0.2500
Nature : Local Minimum
Reason : Both eigenvalues are positive

```
%% ============================================================
% GLOBAL MINIMUM IDENTIFICATION
%% ============================================================
[min_value, ~] = min(local_min_values);
global_min_points = local_min_points(local_min_values == min_value, :);

fprintf('Local Minimum Points:\n');
for i = 1:size(local_min_points,1)
    fprintf('    (%.4f , %.4f)\n', local_min_points(i,1), local_min_points(i,2));
end
```

```
Local Minimum Points:
(0.7071 , 1.0000)
(0.7071 , -1.0000)
(-0.7071 , 1.0000)
(-0.7071 , -1.0000)
```

```
fprintf('\nGlobal Minimum Value: %.4f\n', min_value);
```

```
Global Minimum Value: -0.2500
```

```
fprintf('Global Minimum Points:\n');
for i = 1:size(global_min_points,1)
    fprintf('    (%.4f , %.4f)\n', global_min_points(i,1), global_min_points(i,2));
end
```

```
Global Minimum Points:

(0.7071 , 1.0000)
(0.7071 , -1.0000)
(-0.7071 , 1.0000)
(-0.7071 , -1.0000)
```

2. Gradient descent

Parameters used:

*Learning rate (η) = 0.1000
*Initial position = (0.5000 , 0.5000)

```
%% ============================================================
% DESCRIPTION OF PARAMETER CHOICES AND TERMINATION CONDITION
%
% The learning rate was selected as η = 0.1 to ensure stable and
% gradual convergence without oscillations.
%
% The initial starting point (0.5, 0.5) was chosen to clearly
% demonstrate the movement of the algorithm toward the global
% minimum.
%
% A fixed iteration count of three was used as the termination
```

```
% condition to facilitate manual verification and illustrate
% the optimization process.
%
% Rounding to four decimal places was applied for numerical
% clarity and consistency with manual calculations.
%% ========================================================
% OBJECTIVE FUNCTION
% Defines the nonlinear function f(x,y) to be minimized using
% Gradient Descent.
%% ========================================================

f = @(x,y) x.^4 + y.^4 - x.^2 - 2*y.^2 + 1;

%% ========================================================
% GRADIENT DEFINITION
% These represent the partial derivatives of f(x,y) with respect
% to x and y, which determine the descent direction.
%% ========================================================

dfx = @(x) 4*x.^3 - 2*x;
dfy = @(y) 4*y.^3 - 4*y;

%% Parameters
alpha = 0.1;
iterations = 3;

%% Initial position
x = round(0.5,4);
y = round(0.5,4);

%% Store path for plotting
x_path = x;
y_path = y;

%% Print parameters
fprintf('Gradient Descent Parameters:\n');
fprintf('Learning rate (η)    = %.4f\n', alpha);
fprintf('Initial position          = (%.4f , %.4f)\n', x, y);
fprintf('Number of iterations    = %d\n\n', iterations);
```

```
Gradient Descent Parameters:

Learning rate (η) = 0.1000
Initial position = (0.5000 , 0.5000)
Number of iterations = 3
```

```
fprintf('Gradient Descent with Rounding (4-decimal)\n\n');
%% Gradient Descent iterations
for k = 0:iterations-1

    fprintf('Iteration %d:\n', k);
    fprintf('Current Position : x = %.4f , y = %.4f\n', x, y);

    %% Gradient (notation gx, gy)
```

```
    g_x = dfx(x);
    g_y = dfy(y);

    fprintf('Gradient Calculation:\n');
    fprintf('    g_x = 4*(%.4f)^3 - 2*(%.4f) = %.4f\n', x, x, g_x);
    fprintf('    g_y = 4*(%.4f)^3 - 4*(%.4f) = %.4f\n', y, y, g_y);

    %% Update step
    x_new = x - alpha * g_x;
    y_new = y - alpha * g_y;

    fprintf('Update Step:\n');
    fprintf('    x_new = %.4f - %.4f*(%.4f) = %.4f\n', x, alpha, g_x, x_new);
    fprintf('    y_new = %.4f - %.4f*(%.4f) = %.4f\n', y, alpha, g_y, y_new);

    %% Round and carry forward
    x = round(x_new,4);
    y = round(y_new,4);

    %% Store path
    x_path = [x_path x];
    y_path = [y_path y];

    fprintf('Updated Position : x = %.4f , y = %.4f\n\n', x, y);
end
```

```
Gradient Descent with Rounding (4-decimal)

Iteration 0:
Current Position : x = 0.5000 , y = 0.5000
Gradient Calculation:
g_x = 4*(0.5000)^3 - 2*(0.5000) = -0.5000
g_y = 4*(0.5000)^3 - 4*(0.5000) = -1.5000
Update Step:
x_new = 0.5000 - 0.1000*(-0.5000) = 0.5500
y_new = 0.5000 - 0.1000*(-1.5000) = 0.6500
Updated Position : x = 0.5500 , y = 0.6500

Iteration 1:
Current Position : x = 0.5500 , y = 0.6500
Gradient Calculation:
g_x = 4*(0.5500)^3 - 2*(0.5500) = -0.4345
g_y = 4*(0.6500)^3 - 4*(0.6500) = -1.5015
Update Step:
x_new = 0.5500 - 0.1000*(-0.4345) = 0.5935
y_new = 0.6500 - 0.1000*(-1.5015) = 0.8002
Updated Position : x = 0.5935 , y = 0.8002

Iteration 2:
Current Position : x = 0.5935 , y = 0.8002
Gradient Calculation:
g_x = 4*(0.5935)^3 - 2*(0.5935) = -0.3508
g_y = 4*(0.8002)^3 - 4*(0.8002) = -1.1513
Update Step:
x_new = 0.5935 - 0.1000*(-0.3508) = 0.6286
```

```
y_new = 0.8002 - 0.1000*(-1.1513) = 0.9153
Updated Position : x = 0.6286 , y = 0.9153
```

```
%% SURFACE PLOT FOR VISUALIZATION
[X,Y] = meshgrid(-2:0.1:2, -2:0.1:2);
Z = f(X,Y);

%% ============================================================
% FIGURE 1: FRONT VIEW
% Shows overall surface shape and optimization trajectory
%% ============================================================
figure
surf(X,Y,Z,'EdgeColor','none')
shading interp
hold on
% Gradient Descent path with direction
plot3(x_path, y_path, f(x_path,y_path), ...
        'k>-', 'LineWidth', 3, 'MarkerSize', 10)

% Initial point
plot3(x_path(1), y_path(1), f(x_path(1),y_path(1)), ...
        'yo', 'MarkerSize', 12, 'LineWidth', 2)

% Final point
plot3(x_path(end), y_path(end), f(x_path(end),y_path(end)), ...
        'go', 'MarkerSize', 12, 'LineWidth', 2)

xlabel('x')
ylabel('y')
zlabel('f(x,y)')
title('Front View: Gradient Descent on Surface')
legend('Surface','Gradient Descent Path','Initial Point','Final Point')
view(45,30)        % front / 3D view
grid on
hold off
```

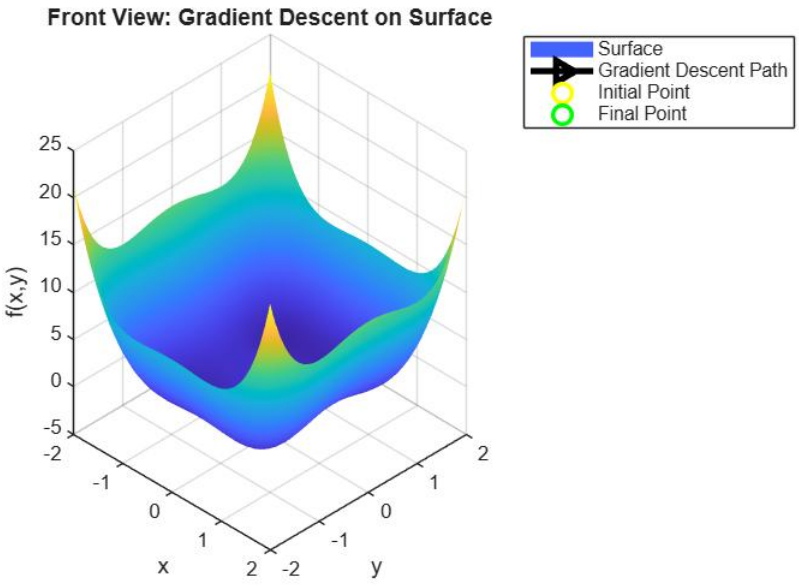


Figure 1

```
%% ================================================================
% FIGURE 2: TOP VIEW
% Shows direction of movement in direction of global optimum from initial position
%% ================================================================
figure
surf(X,Y,Z,'EdgeColor','none')
shading interp
hold on

% Gradient Descent path with direction
plot3(x_path, y_path, f(x_path,y_path), ...
        'k->', 'LineWidth',1.5, 'MarkerSize', 10)

% Initial point
plot3(x_path(1), y_path(1), f(x_path(1),y_path(1)), ...
        'yo', 'MarkerSize', 12, 'LineWidth', 2)

% Final point
plot3(x_path(end), y_path(end), f(x_path(end),y_path(end)), ...
        'go', 'MarkerSize', 12, 'LineWidth', 2)

xlabel('x')
ylabel('y')
zlabel('f(x,y)')
title('Near Top View: Direction Toward Global Minimum')
legend('Surface','Gradient Descent Path','Initial Point','Final Point')
view(45,70)         % near top view
grid on
hold off
```

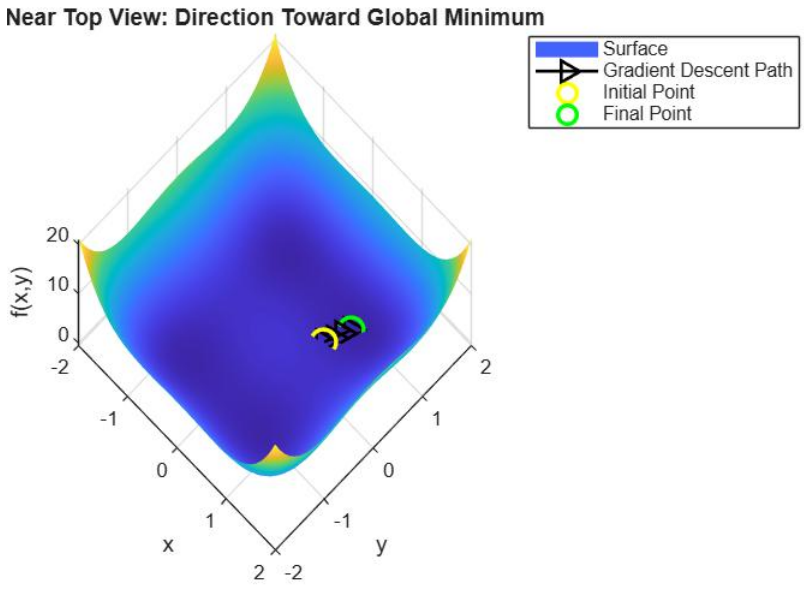


Figure 2

3. Gradient descent with momentum

Parameters used:

*Learning rate (η) = 0.1000
*Momentum coefficient (β) = 0.9000
*Initial position = (0.5000 , 0.5000)
*Initial velocity = (0.0000 , 0.0000)

```
%% ========================================================
% DESCRIPTION OF PARAMETER CHOICES AND TERMINATION CONDITION
%
% The learning rate η = 0.1 is chosen to control step size and
% ensure stability,
% while the momentum coefficient β = 0.9 helps accelerate
% convergence by incorporating information from previous steps (past
% velocity)
%
% The initial point (0.5, 0.5) is selected to clearly visualize
% the movement of the algorithm toward the global minimum.
%
% A fixed iteration count of three is used as the termination
% condition to allow manual verification and demonstration of
% the optimization process rather than full convergence.
%
% Rounding to four decimal places is applied for numerical clarity.
%% ========================================================

%% ========================================================
% OBJECTIVE FUNCTION
%% ========================================================
f = @(x,y) x.^4 + y.^4 - x.^2 - 2*y.^2 + 1;

%% ========================================================
```

```
% GRADIENT DEFINITIONS
%% ================================================================
dfx = @(x) 4*x.^3 - 2*x;
dfy = @(y) 4*y.^3 - 4*y;

%% ================================================================
% ALGORITHM PARAMETERS
%% ================================================================
alpha = 0.1;          % learning rate
beta  = 0.9;          % momentum coefficient
iterations = 3;     % termination condition

%% ================================================================
% INITIAL CONDITIONS
%% ================================================================
x  = round(0.5,4);
y  = round(0.5,4);
vx = round(0,4);
vy = round(0,4);

%% ================================================================
% PATH STORAGE
%% ================================================================
x_path = x;
y_path = y;

%% ================================================================
% PRINT INITIAL PARAMETERS
%% ================================================================
fprintf('Gradient Descent with Momentum Parameters:\n');
fprintf('Learning rate (η)              = %.4f\n', alpha);
fprintf('Momentum coefficient (β) = %.4f\n', beta);
fprintf('Initial position               = (%.4f , %.4f)\n', x, y);
fprintf('Initial velocity               = (%.4f , %.4f)\n', vx, vy);
fprintf('Termination condition      = Fixed iterations (%d)\n\n', iterations);
```

```
Gradient Descent with Momentum Parameters:

Learning rate (η) = 0.1000
Momentum coefficient (β) = 0.9000
Initial position = (0.5000 , 0.5000)
Initial velocity = (0.0000 , 0.0000)
Termination condition = Fixed iterations (3)
```

```
%% ================================================================
% MOMENTUM ITERATIONS
%% ================================================================
for k = 0:iterations-1

    fprintf('Iteration %d\n', k);
    fprintf('Current Position : x = %.4f , y = %.4f\n', x, y);
    fprintf('Current Velocity : vx = %.4f , vy = %.4f\n', vx, vy);

    % Gradient computation
```

```
    g_x = dfx(x);
    g_y = dfy(y);

    %% Gradient calculation with substitution
    fprintf('Gradient Calculation:\n');
    fprintf('    g_x = 4*(%.4f)^3 - 2*(%.4f) = %.4f\n', x, x, g_x);
    fprintf('    g_y = 4*(%.4f)^3 - 4*(%.4f) = %.4f\n', y, y, g_y);

    %% Momentum velocity update
    vx_new = beta * vx + alpha * g_x;
    vy_new = beta * vy + alpha * g_y;

    fprintf('Velocity Update:\n');
    fprintf('    vx = %.4f*(%.4f) + %.4f*(%.4f) = %.4f\n', ...
            beta, vx, alpha, g_x, vx_new);
    fprintf('    vy = %.4f*(%.4f) + %.4f*(%.4f) = %.4f\n', ...
            beta, vy, alpha, g_y, vy_new);

    % Rounding velocity
    vx = round(vx_new,4);
    vy = round(vy_new,4);

    %% Position update
    x_new = x - vx;
    y_new = y - vy;

    fprintf('Position Update:\n');
    fprintf('    x_new = %.4f - %.4f = %.4f\n', x, vx, x_new);
    fprintf('    y_new = %.4f - %.4f = %.4f\n', y, vy, y_new);

    % Rounding position
    x = round(x_new,4);
    y = round(y_new,4);

    % Store trajectory
    x_path = [x_path x];
    y_path = [y_path y];

    fprintf('Updated Velocity : vx = %.4f , vy = %.4f\n', vx, vy);
    fprintf('Updated Position : x = %.4f , y = %.4f\n\n', x, y);
end
```

```
Iteration 0
Current Position : x = 0.5000 , y = 0.5000
Current Velocity : vx = 0.0000 , vy = 0.0000
Gradient Calculation:
g_x = 4*(0.5000)^3 - 2*(0.5000) = -0.5000
g_y = 4*(0.5000)^3 - 4*(0.5000) = -1.5000
Velocity Update:
vx = 0.9000*(0.0000) + 0.1000*(-0.5000) = -0.0500
vy = 0.9000*(0.0000) + 0.1000*(-1.5000) = -0.1500
Position Update:
x_new = 0.5000 - -0.0500 = 0.5500
y_new = 0.5000 - -0.1500 = 0.6500
Updated Velocity : vx = -0.0500 , vy = -0.1500
```

```
Updated Position : x = 0.5500 , y = 0.6500


Iteration 1
Current Position : x = 0.5500 , y = 0.6500
Current Velocity : vx = -0.0500 , vy = -0.1500
Gradient Calculation:
g_x = 4*(0.5500)^3 - 2*(0.5500) = -0.4345
g_y = 4*(0.6500)^3 - 4*(0.6500) = -1.5015
Velocity Update:
vx = 0.9000*(-0.0500) + 0.1000*(-0.4345) = -0.0885
vy = 0.9000*(-0.1500) + 0.1000*(-1.5015) = -0.2852
Position Update:
x_new = 0.5500 - -0.0885 = 0.6385
y_new = 0.6500 - -0.2852 = 0.9352
Updated Velocity : vx = -0.0885 , vy = -0.2852
Updated Position : x = 0.6385 , y = 0.9352


Iteration 2
Current Position : x = 0.6385 , y = 0.9352
Current Velocity : vx = -0.0885 , vy = -0.2852
Gradient Calculation:
g_x = 4*(0.6385)^3 - 2*(0.6385) = -0.2358
g_y = 4*(0.9352)^3 - 4*(0.9352) = -0.4691
Velocity Update:
vx = 0.9000*(-0.0885) + 0.1000*(-0.2358) = -0.1032
vy = 0.9000*(-0.2852) + 0.1000*(-0.4691) = -0.3036
Position Update:
x_new = 0.6385 - -0.1032 = 0.7417
y_new = 0.9352 - -0.3036 = 1.2388
Updated Velocity : vx = -0.1032 , vy = -0.3036
Updated Position : x = 0.7417 , y = 1.2388
```

```
%% ========================================================
% SURFACE PLOT FOR VISUALIZATION
%% ========================================================
[X,Y] = meshgrid(-2:0.1:2, -2:0.1:2);
Z = f(X,Y);

%% ========================================================
% FIGURE 3: FRONT VIEW
% Shows surface shape and momentum-based trajectory
%% ========================================================
figure
surf(X,Y,Z,'EdgeColor','none')
colormap parula
hold on

plot3(x_path, y_path, f(x_path,y_path), ...
        'k>-', 'LineWidth', 3, 'MarkerSize', 10)

plot3(x_path(1), y_path(1), f(x_path(1),y_path(1)), ...
        'ro', 'MarkerSize', 12, 'LineWidth', 2)
```

```
plot3(x_path(end), y_path(end), f(x_path(end),y_path(end)), ...
        'go', 'MarkerSize', 12, 'LineWidth', 2)

xlabel('x')
ylabel('y')
zlabel('f(x,y)')
title('Front View: Gradient Descent with Momentum')
legend('Surface','Momentum Path','Initial Point','Final Point')
view(45,30)
grid on
hold off
```

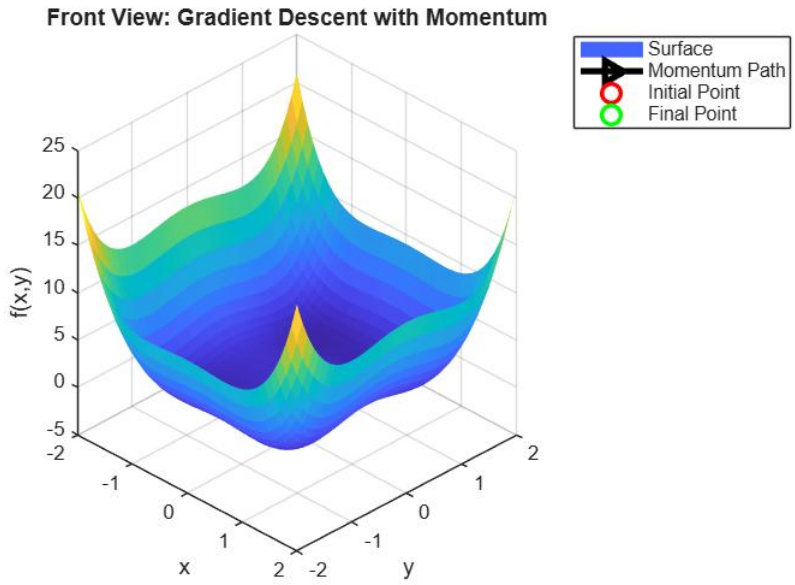


Figure 3

```
%% ========================================================================
% FIGURE 4: TOP VIEW
%Shows direction of movement in direction of global optimum from initial position
%% ========================================================================
figure
surf(X,Y,Z,'EdgeColor','none')
colormap parula
hold on

plot3(x_path, y_path, f(x_path,y_path), ...
        'k>-', 'LineWidth',1.5, 'MarkerSize', 10)

plot3(x_path(1), y_path(1), f(x_path(1),y_path(1)), ...
        'ro', 'MarkerSize', 12, 'LineWidth', 2)

plot3(x_path(end), y_path(end), f(x_path(end),y_path(end)), ...
        'go', 'MarkerSize', 12, 'LineWidth', 2)

xlabel('x')
ylabel('y')
zlabel('f(x,y)')
title('       Top View: Momentum Direction Toward Global Minimum')
```

```
legend('Surface','Momentum Path','Initial Point','Final Point')
view(45,70)
grid on
hold off
```

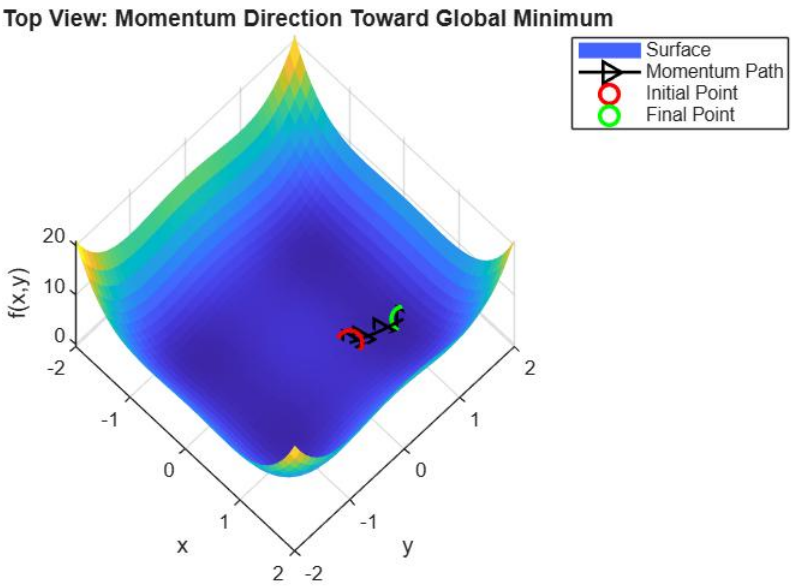


Figure 4

5. ADAGRAD

Parameters used:

*Learning rate (η) = 0.8000
*Epsilon (ϵ) = 1.0e-08
*Initial position = (0.5000 , 0.5000)
*Initial accumulators (vx,vy) = (0.0000 , 0.0000)

```
%% ============================================================
% DESCRIPTION OF PARAMETER CHOICES AND TERMINATION CONDITION
%
% The learning rate η = 0.8 is chosen to allow noticeable progress
% when combined with Adagrad's adaptive step size mechanism.
% As the accumulated gradients grow, the effective learning rate
% automatically decreases, ensuring stability.
%
% A small constant ε = 1e-8 is added in the denominator to avoid
% division by zero during the adaptive update.
%
% The initial starting point (0.5, 0.5) is selected to clearly
% demonstrate the movement of the algorithm toward the global
% minimum.
%
% A fixed iteration count of three is used as the termination
% condition to facilitate manual verification and illustrate the
% optimization process rather than full convergence.
%
% Rounding to four decimal places is applied for numerical clarity.
%% ============================================================
```

```
%% ========================================================
% OBJECTIVE FUNCTION
%% ========================================================
f = @(x,y) x.^4 + y.^4 - x.^2 - 2*y.^2 + 1;

%% ========================================================
% GRADIENT DEFINITIONS (MANUALLY DERIVED)
%% ========================================================
dfx = @(x) 4*x.^3 - 2*x;
dfy = @(y) 4*y.^3 - 4*y;

%% ========================================================
% ALGORITHM PARAMETERS
%% ========================================================
alpha = 0.8;          % learning rate
epsilon = 1e-8;     % numerical stability constant
iterations = 3;     % termination condition

%% ========================================================
% INITIAL CONDITIONS
%% ========================================================
x   = round(0.5,4);
y   = round(0.5,4);
vx = round(0,4);     % accumulator for x
vy = round(0,4);     % accumulator for y

%% ========================================================
% PATH STORAGE
%% ========================================================
x_path = x;
y_path = y;

%% ========================================================
% PRINT INITIAL PARAMETERS
%% ========================================================
fprintf('Adagrad Parameters:\n');
fprintf('Learning rate (η)                    = %.4f\n', alpha);
fprintf('Epsilon (ϵ)                          = %.1e\n', epsilon);
fprintf('Initial position                   = (%.4f , %.4f)\n', x, y);)
fprintf('Initial accumulators (vx,vy) = (%.4f , %.4f)\n', vx, vy);
fprintf('Termination condition              = Fixed iterations (%d)\n\n', iterations);
```

```
Adagrad Parameters:

Learning rate (η) = 0.8000
Epsilon (ϵ) = 1.0e-08
Initial position = (0.5000 , 0.5000)
Initial accumulators (vx,vy) = (0.0000 , 0.0000)
Termination condition = Fixed iterations (3)
```

```
%% ========================================================
% ADAGRAD ITERATIVE LOOP
%% ========================================================
```

```
for k = 0:iterations-1

    fprintf('\nIteration %d\n', k);

    % Current position
    fprintf('Current Position:\n');
    fprintf('    x = %.4f , y = %.4f\n', x, y);

    % Gradient computation
    g_x = dfx(x);
    g_y = dfy(y);

    fprintf('Gradient Calculation:\n');
    fprintf('    g_x = 4*(%.4f)^3 - 2*(%.4f) = %.4f\n', x, x, g_x);
    fprintf('    g_y = 4*(%.4f)^3 - 4*(%.4f) = %.4f\n', y, y, g_y);

    % Accumulator update
    vx_new = vx + g_x^2;
    vy_new = vy + g_y^2;

    fprintf('Accumulator Update:\n');
    fprintf('    vx_new = %.4f + (%.4f)^2 = %.4f\n', vx, g_x, vx_new);
    fprintf('    vy_new = %.4f + (%.4f)^2 = %.4f\n', vy, g_y, vy_new);

    % Round accumulators
    vx = round(vx_new,4);
    vy = round(vy_new,4);

    % Position update (Adagrad rule)
    x_new = x - (alpha / (sqrt(vx) + epsilon)) * g_x;
    y_new = y - (alpha / (sqrt(vy) + epsilon)) * g_y;

    fprintf('Update Equations:\n');
    fprintf('    x_new = %.4f - (%.4f/(sqrt(%.4f)))*%.4f = %.4f\n', ...
            x, alpha, vx, g_x, x_new);
    fprintf('    y_new = %.4f - (%.4f/(sqrt(%.4f)))*%.4f = %.4f\n', ...
            y, alpha, vy, g_y, y_new);

    % Rounding and carry forward
    x = round(x_new,4);
    y = round(y_new,4);

    fprintf('Updated Position:\n');
    fprintf('    x = %.4f , y = %.4f\n\n', x, y);

    % Store trajectory
    x_path = [x_path x];
    y_path = [y_path y];
end
```

```
Iteration 0
Current Position:
x = 0.5000 , y = 0.5000
Gradient Calculation:
g_x = 4*(0.5000)^3 - 2*(0.5000) = -0.5000
```

```
g_y = 4*(0.5000)^3 - 4*(0.5000) = -1.5000
Accumulator Update:
vx_new = 0.0000 + (-0.5000)^2 = 0.2500
vy_new = 0.0000 + (-1.5000)^2 = 2.2500
Update Equations:
x_new = 0.5000 - (0.8000/(sqrt(0.2500)))*-0.5000 = 1.3000
y_new = 0.5000 - (0.8000/(sqrt(2.2500)))*-1.5000 = 1.3000
Updated Position:
x = 1.3000 , y = 1.3000


Iteration 1
Current Position:
x = 1.3000 , y = 1.3000
Gradient Calculation:
g_x = 4*(1.3000)^3 - 2*(1.3000) = 6.1880
g_y = 4*(1.3000)^3 - 4*(1.3000) = 3.5880
Accumulator Update:
vx_new = 0.2500 + (6.1880)^2 = 38.5413
vy_new = 2.2500 + (3.5880)^2 = 15.1237
Update Equations:
x_new = 1.3000 - (0.8000/(sqrt(38.5413)))*6.1880 = 0.5026
y_new = 1.3000 - (0.8000/(sqrt(15.1237)))*3.5880 = 0.5619
Updated Position:
x = 0.5026 , y = 0.5619


Iteration 2
Current Position:
x = 0.5026 , y = 0.5619
Gradient Calculation:
g_x = 4*(0.5026)^3 - 2*(0.5026) = -0.4974
g_y = 4*(0.5619)^3 - 4*(0.5619) = -1.5380
Accumulator Update:
vx_new = 38.5413 + (-0.4974)^2 = 38.7887
vy_new = 15.1237 + (-1.5380)^2 = 17.4890
Update Equations:
x_new = 0.5026 - (0.8000/(sqrt(38.7887)))*-0.4974 = 0.5665
y_new = 0.5619 - (0.8000/(sqrt(17.4890)))*-1.5380 = 0.8561
Updated Position:
x = 0.5665 , y = 0.8561
```

```
%% ============================================================
% SURFACE PLOT VISUALIZATION
%% ============================================================
[X,Y] = meshgrid(-2:0.1:2, -2:0.1:2);
Z = f(X,Y);

%% ============================================================
% FIGURE 5: FRONT VIEW
% Shows surface shape and Adagrad trajectory
%% ============================================================
figure
surf(X,Y,Z,'EdgeColor','none')
colormap parula
```

```
hold on

plot3(x_path, y_path, f(x_path,y_path), ...
        'k->', 'LineWidth', 3, 'MarkerSize', 10)

plot3(x_path(1), y_path(1), f(x_path(1),y_path(1)), ...
        'yo', 'MarkerSize', 12, 'LineWidth', 2)

plot3(x_path(end), y_path(end), f(x_path(end),y_path(end)), ...
        'go', 'MarkerSize', 12, 'LineWidth', 2)

xlabel('x')
ylabel('y')
zlabel('f(x,y)')
title('Front View: Adagrad Optimization')
legend('Surface','Adagrad Path','Initial Point','Final Point')
view(45,30)
grid on
hold off
```

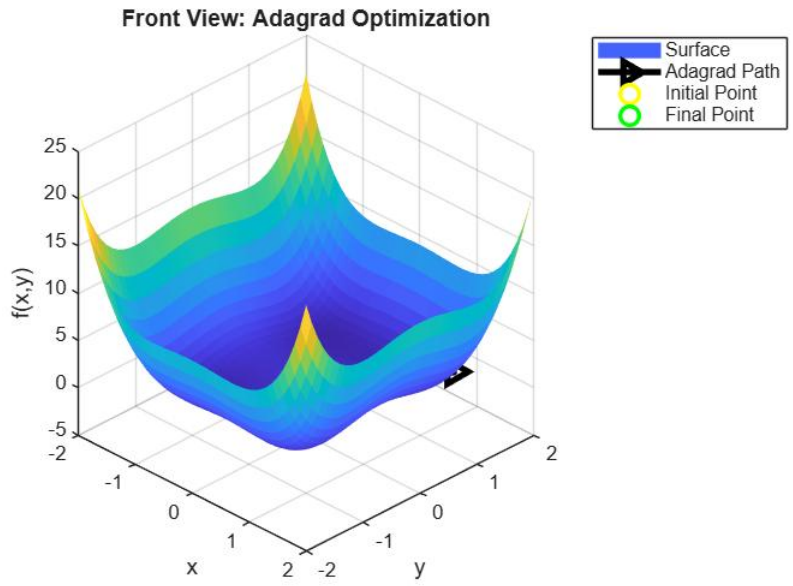


Figure 5

```
%% ============================================================
% FIGURE 6: TOP VIEW
%Shows direction of movement in direction of global optimum from initial position
%% ============================================================

figure
hSurf = surf(X,Y,Z,'EdgeColor','none');
colormap parula
hold on

% Path line
hPath = plot3(x_path, y_path, f(x_path,y_path), ...
                    'k-', 'LineWidth', 1);

% Direction arrows
```

```
hDir = quiver3( ...
            x_path(1:end-1), ...
            y_path(1:end-1), ...
            f(x_path(1:end-1),y_path(1:end-1)), ...
            diff(x_path), ...
            diff(y_path), ...
            zeros(size(diff(x_path))), ...
            0, 'k', 'LineWidth',1, 'MaxHeadSize', 0.8);

% Initial point
hInit = plot3(x_path(1), y_path(1), f(x_path(1),y_path(1)), ...
                    'yo', 'MarkerSize', 12, 'LineWidth', 2);

% Final point
hFinal = plot3(x_path(end), y_path(end), f(x_path(end),y_path(end)), ...
                     'go', 'MarkerSize', 12, 'LineWidth', 2);

xlabel('x')
ylabel('y')
zlabel('f(x,y)')
title('Near Top View: Direction of Movement Toward Final Point')

legend([hSurf, hPath, hDir, hInit, hFinal], ...
        {'Surface','Path','Direction','Initial Point','Final Point'})

view(45,70)
grid on
hold off
```

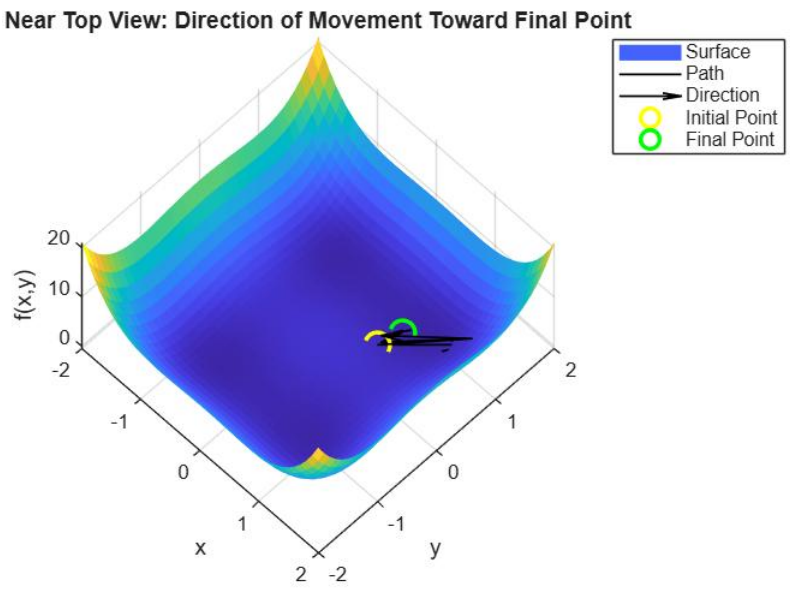


Figure 6